

OUTBREAK

STOCHASTIC SURVIVAL LAB

Ficha técnica del repositorio · Proyecto 1 de Modelación y Simulación



Repositorio oficial

https://github.com/DanielBarillasM/Proyecto-1_Grupo-1_Modelacion-y-Simulacion_Sec-30

Pablo Daniel Barillas Moreno · Jorge Palacios
Andrés Rafael Chivalán · Javier Andrés Chen

Idea central: Simular y comparar patrones de aparición de enemigos mediante Poisson y una renovación Polar-lognormal, cuantificando su efecto sobre congestión y supervivencia.

1 · IDENTIDAD Y ALCANCE

Presentación del repositorio

Outbreak es una aplicación académica reproducible que integra generación de variables aleatorias continuas, simulación computacional, inferencia Monte Carlo y visualización tridimensional. La narrativa zombi contextualiza un sistema real de software: el subsistema que decide cuándo aparecen enemigos y cómo esa presión modifica el balance de una misión.

Curso	CC2017 — Modelación y Simulación, Sección 30
Tipo	Aplicación web interactiva y laboratorio estadístico
Modelo principal	Proceso de Poisson homogéneo
Contraste	Renovación lognormal generada con Marsaglia Polar
Cantidad de interés	Probabilidad de supervivencia y congestión
Interfaz	Python, Streamlit y Plotly Mesh3d
Licencia	MIT

Pregunta de investigación

Pregunta: ¿Cómo cambian la congestión, la supervivencia y el costo computacional cuando se conserva la tasa media, pero se modifica el método y la distribución de los interarribos?

Capacidades principales

- Configurar tasa, duración, semilla, combate y variabilidad Polar.
- Generar calendarios Poisson por transformada inversa.
- Generar normales con Marsaglia Polar e interarribos lognormales positivos.
- Explorar una misión mediante una escena 3D y telemetría temporal.
- Validar generadores con pruebas formales y momentos observados.
- Comparar modelos con diseño pareado, intervalos y pruebas estadísticas.

Dos pipelines de generación



Criterio	Poisson-exponencial	Polar-lognormal
Interarribo	Exponencial	Lognormal
Método	Transformada inversa	Marsaglia + transformación
Media	$1/\lambda$	$1/\lambda$
CV base	1	0.45
Conteo	Poisson exacto	Renovación
Memoria	Sin memoria	No exponencial

Ecuaciones esenciales

$$\begin{aligned}
 N(t) &\sim \text{Poisson}(\lambda t) & E[N(t)] &= \text{Var}[N(t)] = \lambda t \\
 \Delta &= -\ln(U)/\lambda & t_i &= t_{i-1} + \Delta_i \\
 Z &= V \cdot \sqrt{-2 \ln(S)/S} & S &= V_1^2 + V_2^2 \\
 \Delta &= \exp(\mu + \sigma Z) & \sigma^2 &= \ln(1+c^2) \\
 \mu &= \ln(1/\lambda) - \sigma^2/2 & E[\Delta] &= 1/\lambda
 \end{aligned}$$

Distinción esencial: Marsaglia Polar no es otro proceso de Poisson. Produce normales que se transforman en interarribos lognormales; por ello el modelo alternativo es un proceso de renovación.

3 · ARQUITECTURA Y EJECUCIÓN

Organización del proyecto

```

zombie_poisson_streamlit/
├── App/           interfaz, motor y visualizaciones
├── assets/        imágenes y figuras reproducibles
├── presentation/  presentación y guion de exposición
├── report/        informe, datos y generador de resultados
├── requirements/  dependencias fijadas
└── tests/         pruebas automatizadas

```

Responsabilidades del código

simulation.py	Generadores, combate, Monte Carlo e inferencia pareada
visuals.py	Escena 3D, histogramas, Q-Q y gráficos comparativos
app.py	Formulario, estado, caché, navegación y explicación
generate_report_assets.py	Regeneración de CSV, figuras y macros LaTeX
test_simulation.py	Doce pruebas deterministas y estadísticas

Inicio rápido

```

git clone https://github.com/DanielBarillasM/Proyecto-1_Grupo-1_Modelacion-y-Simulacion_Sec-30.git
cd Proyecto-1_Grupo-1_Modelacion-y-Simulacion_Sec-30/zombie_poisson_streamlit
python -m pip install -r requirements/requirements.txt
python -m streamlit run App/app.py

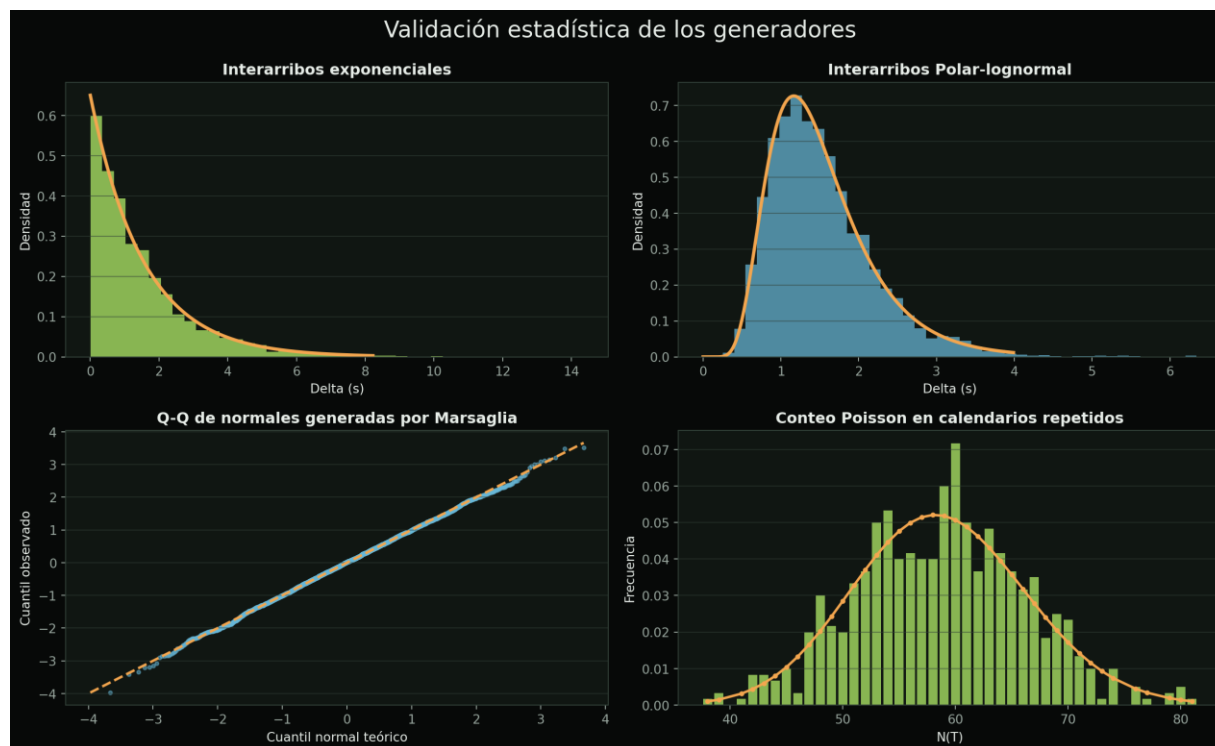
```

Flujo recomendado dentro de la aplicación:

- Configurar el escenario y pulsar SIMULAR MISIÓN.
- Recorrer el tiempo de la escena 3D y observar la telemetría.
- Ejecutar la validación formal de los generadores.
- Ejecutar la comparación Poisson vs. Polar y revisar efectos pareados.
- Explorar el laboratorio Monte Carlo y la sensibilidad a la tasa.

4 · VALIDACIÓN Y RESULTADOS

Evidencia estadística reproducible



La validación usa muestras fijas sin censura. Incluye KS para exponencial, normal y lognormal; correlación de rezago uno; dispersión de conteos; aceptación Polar contra $\pi/4$; momentos y benchmark. No rechazar indica compatibilidad con el modelo, no una prueba absoluta de verdad.

Escenario base: 400 partidas por modelo

Métrica	Poisson	Polar-lognormal
Supervivencia	96.5 %	100.0 %
IC Wilson 95 %	[94.2, 97.9] %	[99.0, 100.0] %
Tiempo medio	88.528 s	90.000 s
Llegadas medias	57.133	57.778
Pico concurrente	9.483	6.483

Lectura: El efecto de supervivencia Poisson – Polar fue –3.5 puntos porcentuales, con IC bootstrap [–5.5, –1.8] y McNemar exacta $p = 0.000122$. Poisson produjo aproximadamente tres enemigos más en el pico concurrente.

Poisson permanece como modelo principal por correspondencia con el conteo, parsimonia, costo e interpretación; no porque garantice mayor supervivencia. Sin telemetría real, la elección es condicional a los supuestos.

5 · ENTREGABLES, EQUIPO Y USO ACADÉMICO

Material incluido

- Aplicación funcional en Streamlit con escena 3D interactiva.
- README con instalación, teoría, arquitectura y uso.
- Informe LaTeX compilado a PDF con resultados reproducibles.
- Presentación HTML de 25 diapositivas, operativa sin conexión.
- Guion LaTeX dividido entre los cuatro integrantes.
- CSV y figuras regenerables para auditoría.
- Doce pruebas automatizadas del motor y la estadística.

Integrantes y reparto sugerido

Integrante	Diapositivas	Tema
Pablo Daniel Barillas Moreno	1–6	Problema, objetivos y Poisson
Jorge Palacios	7–13	Polar, validación y modelo
Andrés Rafael Chivalán	14–19	Software, Monte Carlo y resultados
Javier Andrés Chen	20–25	Interpretación, demo y cierre

Acceso

Página del repositorio: https://github.com/DanielBarillasM/Proyecto-1_Grupo-1_Modelacion-y-Simulacion_Sec-30

URL de clonación: https://github.com/DanielBarillasM/Proyecto-1_Grupo-1_Modelacion-y-Simulacion_Sec-30.git

Uso académico: La licencia MIT permite revisar, ejecutar y adaptar el software conservando el aviso de copyright. Los resultados deben interpretarse dentro de los supuestos y parámetros declarados.